

Instalación local en la intranet de la estación

Guía para desplegar el sistema completo (PostgreSQL, API y web) en un servidor de la red local, con el escenario de dimensionamiento acordado: hasta 40 PCs cliente y unos 20 usuarios activos en simultáneo. Complementa a DEPLOY.md, que cubre el despliegue en la nube para la demo.

1. Especificaciones recomendadas del servidor

Recurso	Mínimo funcional	Recomendado (40 PCs / 20 activos)
CPU	2 núcleos	4 a 8 núcleos
RAM	4 GB	16 GB
Disco	60 GB SSD	256 GB SSD
Red	LAN 100 Mbps, IP fija	LAN gigabit, IP fija
Sistema operativo	Windows 10/11 Pro + Docker Desktop	Ubuntu Server 24.04 LTS + Docker Engine
Software	Docker + Docker Compose	Docker + Docker Compose

Las PCs cliente solo necesitan un navegador moderno (Chrome, Edge o Firefox) y estar en la misma red. No se instala nada en ellas. Acceden por la IP del servidor.

Sobre el disco: la base de datos con 5.000 expedientes ronda los 2 GB. Lo que crece son los archivos adjuntos (fotos, huellas, firmas, documentos escaneados), estimados entre 5 y 15 GB en los primeros años. Los 256 GB dan margen de sobra para el histórico completo más los respaldos locales.

2. Qué componentes se levantan

Servicio	Puerto	Contenedor
PostgreSQL 16	5432 (solo interno)	bomberos_pg
API FastAPI	8000	bomberos_api
Web Next.js	3000	proceso Node (contenedor pendiente)

El `docker-compose.yml` de la raíz levanta Postgres y la API. La web todavía no tiene contenedor propio, se ejecuta con Node directamente (paso 5). Al primer arranque la API corre un bootstrap idempotente que aplica los esquemas SQL si la base está vacía y crea el usuario administrador inicial desde variables de entorno.

3. Preparación (una sola vez, en una máquina con internet)

La intranet de la estación no tiene salida a internet, así que las imágenes se preparan afuera y se llevan en un pendrive o disco externo.

En la máquina con internet, dentro de la carpeta del repositorio:

```
# Construir las imágenes
docker compose build
docker pull postgres:16-alpine
docker pull node:20-alpine

# Exportarlas a archivos
docker save -o bomberos_api.tar bomberos-caracas-bd-api
docker save -o postgres16.tar postgres:16-alpine
docker save -o node20.tar node:20-alpine

# Compilar la web para producción
cd apps/web
npm ci
npm run build
```

Copiar al medio externo: el repositorio completo (incluyendo `apps/web/.next` y `apps/web/node_modules`) y los tres archivos `.tar`.

En el servidor de la estación:

```
docker load -i postgres16.tar
docker load -i bomberos_api.tar
docker load -i node20.tar
```

Si el servidor va a ser Windows, instalar Docker Desktop y Node 20 antes (los instaladores también viajan en el pendrive).

4. Configuración

Crear un archivo `.env` en la raíz del repositorio en el servidor:

```
JWT_SECRET_KEY=<cadena aleatoria de 64+ caracteres>
POSTGRES_PASSWORD=<contraseña fuerte, no dejar la de desarrollo>
BOOTSTRAP_ADMIN_USER=admin
BOOTSTRAP_ADMIN_PASSWORD=<contraseña fuerte inicial>
BOOTSTRAP_ADMIN_EMAIL=admin@bomberos.gob.ve
```

Para generar el JWT secret en PowerShell:

```
-join ((48..57)+(65..90)+(97..122) | Get-Random -Count 64 | % {[char]$_})
```

En `apps/web` crear `.env.production`:

```
NEXT_PUBLIC_DEMO_MODE=0
NEXT_PUBLIC_API_URL=http://IP-DEL-SERVIDOR:8000
API_INTERNAL_URL=http://localhost:8000
```

`NEXT_PUBLIC_DEMO_MODE=0` es el interruptor que apaga los datos ficticios y conecta la web contra la API real. `API_INTERNAL_URL` lo usa el proxy interno de la web para hablar con la API sin salir a la red.

5. Arranque

```
# 1. Base de datos y API
docker compose up -d

# 2. Verificar que la API responde
curl http://localhost:8000/health

# 3. Web en modo producción
cd apps/web
npm run start
```

Para que la web arranque sola al reiniciar el servidor en Windows, registrarla como tarea programada al inicio de sesión o usar `nssm` para instalarla como servicio. En Ubuntu, una unidad `systemd`.

Primer ingreso: `http://IP-DEL-SERVIDOR:3000` con el usuario y contraseña del bootstrap. El sistema fuerza el cambio de contraseña en el primer login.

6. Respaldo diario

Programar en el servidor (tarea programada de Windows o cron en Linux):

```
docker exec bomberos_pg pg_dump -U postgres bomberos_caracas >
"D:\respaldos\bomberos_$(Get-Date -Format yyyy-MM-dd).sql"
```

Conservar al menos 14 días de respaldos y copiar semanalmente a un segundo medio (disco externo u otra PC). El respaldo debe incluir también la carpeta de archivos adjuntos cuando el módulo de documentos entre en uso.

7. Qué falta para producción real

En orden de prioridad:

1. Contenedor de la web (Dockerfile de Next.js standalone) para que los tres servicios vivan en el compose y arranquen juntos.
2. Reverse proxy (Caddy o Nginx) como puerta única: un solo puerto de entrada, la API deja de estar expuesta a la LAN y queda listo para TLS.
3. Migración de los datos del sistema legacy (plan en docs/MIGRACION_LEGACY_PLAN.md).
4. Con la proyección de 20 usuarios activos: cache de permisos a Redis y 4 workers de uvicorn, más el endpoint agregado del expediente (detalle en la auditoría del 4 de julio).
5. Scheduler para procesos programados (cierres automáticos, alertas de vencimiento) cuando esas funcionalidades se implementen.

Última actualización: julio 2026